

Entrega N° 1

Descripción General

Desarrollar un servidor que contenga los endpoints y servicios necesarios para gestionar los productos y carritos de compra para tu API.

Requisitos de la Primera Entrega

Desarrollo del Servidor

El servidor debe estar basado en Node.js y Express, y debe escuchar en el puerto 8080. Se deben disponer dos grupos de rutas: `/products` y `/carts`. Estos endpoints estarán implementados con el router de Express, con las siguientes especificaciones:

Rutas para Manejo de Productos (`/api/products/`)

- **GET /:**
 - Debe listar todos los productos de la base de datos.
- **GET /:pid:**
 - Debe traer solo el producto con el id proporcionado.
- **POST /:**
 - Debe agregar un nuevo producto con los siguientes campos:
 - `id`: Number/String (No se manda desde el body, se autogenera para asegurar que nunca se repitan los ids).
 - `title`: String
 - `description`: String
 - `code`: String
 - `price`: Number
 - `status`: Boolean
 - `stock`: Number
 - `category`: String
 - `thumbnails`: Array de Strings (rutas donde están almacenadas las imágenes del producto).

- **PUT /:pid:**
 - Debe actualizar un producto por los campos enviados desde el body. No se debe actualizar ni eliminar el id al momento de hacer la actualización.
- **DELETE /:pid:**
 - Debe eliminar el producto con el pid indicado.

Rutas para Manejo de Carritos (/api/carts/)

- **POST /:**
 - Debe crear un nuevo carrito con la siguiente estructura:
 - id: Number/String (Autogenerado para asegurar que nunca se dupliquen los ids).
 - products: Array que contendrá objetos que representen cada producto.
- **GET /:cid:**
 - Debe listar los productos que pertenecen al carrito con el cid proporcionado.
- **POST /:cid/product/:pid:**
 - Debe agregar el producto al arreglo products del carrito seleccionado, utilizando el siguiente formato:
 - product: Solo debe contener el ID del producto.
 - quantity: Debe contener el número de ejemplares de dicho producto (se agregará de uno en uno).
 - Si un producto ya existente intenta agregarse, se debe incrementar el campo quantity de dicho producto.

Persistencia de la Información

- La persistencia se implementará utilizando el sistema de archivos, donde los archivos products.json y carts.json respaldarán la información.
- Se debe utilizar el ProductManager desarrollado en el desafío anterior y crear un CartManager para gestionar el almacenamiento de estos archivos JSON.
- Nota: No es necesario realizar ninguna implementación visual, todo el flujo se puede realizar por Postman o por el cliente de tu preferencia.

Formato del Entregable

- Proporcionar un enlace al repositorio de GitHub con el proyecto completo, sin la carpeta

node_modules. Criterios de evaluación

ENTREGAR LINK DEL REPO DE GITHUB publico*** Nosotros usamos insomnia**

EXPLICACION PROF EN CLASE: SEMANA 4 – PARCTICA ENTREGABLE explico entrega 1

```
04_semana > practica-entregable > ① README.md > abc # Entrega N°1: API para gestión de productos y carritos > abc ## Requisitos técnicos
1  # Entrega N°1: API para gestión de productos y carritos
2
3  ## Objetivo
4
5  Construir un servidor (API) básico usando **Node.js + Express** que permita:
6  - **Administrar productos** (crear, leer, actualizar, eliminar).
7  - **Administrar carritos de compra** (crear carrito, ver productos en un carrito, agregar productos a un carrito).
8
9  Los datos se guardarán en archivos `.json` (productos y carritos).
10 No necesitas una interfaz visual; puedes probar todo con **Insomnia** o cualquier cliente HTTP.
11
12 ---
13
14 ## Requisitos técnicos
15
16 > No hace falta tener la version para esta api
17
18 - **Puerto**: `8080`
19 - **Rutas base**:
20   - `/api/products` → manejo de productos
21   - `/api/carts` → manejo de carritos
22 - **Persistencia**: Archivos `products.json` y `carts.json` (se crean automáticamente al guardar datos).
23 - **Herramientas recomendadas**: Node.js, Express, File System (fs) nativo.
```

```
27 ## Endpoints y su comportamiento
28
29 ### 1. Rutas de productos (`/api/products`)
30
31 | Método | Ruta | ¿Qué hace? |
32 |-----|-----|-----|
33 | GET | `/` | Lista **todos** los productos. |
34 | GET | `/:pid` | Muestra el producto con el `id` indicado. |
35 | POST | `/` | Agrega un **nuevo producto**. Los campos obligatorios se envían en el cuerpo (body) de la petición. El `id` se genera automáticamente. |
36 | PUT | `/:pid` | Actualiza los campos de un producto existente (excepto el `id`). |
37 | DELETE | `/:pid` | Elimina el producto con el `id` indicado. |
```

```
#### Estructura de un producto (schema)
| Campo | Tipo | Obligatorio | Observación |
|-----|-----|-----|-----|
| `id` | number o string | automático | Se genera sin repetir (ej: timestamp + random). |
| `title` | string | sí | |
| `description` | string | sí | |
| `code` | string | sí | Código único del producto. |
| `price` | number | sí | |
| `status` | boolean | sí | `true` = habilitado, `false` = deshabilitado. |
| `stock` | number | sí | Cantidad disponible. |
| `category` | string | sí | |
| `thumbnails` | array de strings | no | Rutas (URLs) de imágenes. |
```

```

**Ejemplo de producto (formato JSON):**
```json
{
 "id": "prod_1",
 "title": "Laptop Gamer",
 "description": "Alta performance para data science",
 "code": "LP-100",
 "price": 1250.99,
 "status": true,
 "stock": 15,
 "category": "electrónica",
 "thumbnails": ["http://.../img1.jpg", "http://.../img2.jpg"]
}
```

```

2. Rutas de carritos (`/api/carts`)

| Método | Ruta | ¿Qué hace? |
|--------|----------------------|--|
| POST | `/` | Crea un nuevo carrito vacío . El `id` del carrito se genera automáticamente. |
| GET | `/:cid` | Muestra la lista de productos dentro del carrito con `id` = `cid`. |
| POST | `/:cid/product/:pid` | Agrega el producto `pid` al carrito `cid`. Si el producto ya estaba en el carrito, incrementa su cantidad en 1. |

Estructura de un carrito (schema)

| Campo | Tipo | Descripción |
|------------|------------------|--|
| `id` | number o string | Identificador único del carrito (autogenerado). |
| `products` | array de objetos | Cada objeto representa un producto dentro del carrito. |

Formato de cada objeto dentro de `products`:

| Campo | Tipo | Descripción |
|------------|---------------|---|
| `product` | string/number | `id` del producto (referencia a un producto existente). |
| `quantity` | number | Cantidad de ejemplares de ese producto en el carrito. |

Ejemplo de carrito (formato JSON):

```

```json
{
 "id": "cart_7",
 "products": [
 { "product": "prod_1", "quantity": 3 },
 { "product": "prod_22", "quantity": 1 }
]
}
```

```

> **Importante**

> Al agregar un producto que ya existe en el carrito, **no** crees un nuevo objeto. Solo incrementa `quantity` en 1.

Persistencia - Manejo de archivos

Deberás implementar dos módulos (clases) que trabajen con el sistema de archivos:

- **ProductManager**
 - Responsable de leer/escribir `products.json`.
 - Métodos: `getProducts()`, `getProductById(id)`, `addProduct(product)`, `updateProduct(id, updates)`, `deleteProduct(id)`.
- **CartManager**
 - Responsable de leer/escribir `carts.json`.
 - Métodos: `createCart()`, `getCartById(id)`, `addProductToCart(cartId, productId)`.

Los archivos JSON deben guardarse en el mismo directorio del proyecto (o en una carpeta `/data`). Si no existen al inicio, se crean vacíos.

Relacion de los modelos

- Un **carrito** tiene muchos **productos** (cada uno con una cantidad).
- Un **producto** puede estar en muchos carritos.

¿Qué debes entregar?

- Un enlace a un repositorio **público** de GitHub.
- El repositorio debe contener **todo el código fuente**.
- **No** incluir la carpeta `node_modules`.
- Incluir un archivo `README.md` con instrucciones básicas de instalación y ejecución.

Consejos para empezar (Data Science)

1. ****Inicializa un proyecto Node**:**

```
`npm init -y`
```

Luego instala Express: ``npm install express``

2. ****Estructura sugerida**:**

```
---
```

```
|— src/
|   |— managers/
|   |   |— ProductManager.js
|   |   |— CartManager.js
|   |— routes/
|   |   |— products.routes.js
|   |   |— carts.routes.js
|   |— app.js
|— data/           (se creará automáticamente)
|— package.json
— README.md
```

3. ****Prueba con Postman|Insomnia****

- Envía peticiones a ``http://localhost:8080/api/products/`` y ``http://localhost:8080/api/carts/``.

